

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche, Informatiche e Matematiche
Corso di Laurea Magistrale in Informatica

Relazione per l'esame di

Kernel Hacking

A.A. 2025/2026

Fix di un lockdep warning in XFS: inconsistent softirq lock state in igrab()

dall'analisi del bug syzbot alla sottomissione della patch upstream

Docente

Prof. Paolo Valente
paolo.valente@unimore.it

Candidato

Francesco Caligiuri
Matricola 207688
276621@studenti.unimore.it

Anno Accademico 2025/2026

Indice

1	Introduzione	4
2	Concetti di base	4
2.1	Syzkaller e syzbot	4
2.2	KASAN (Kernel Address Sanitizer)	5
2.3	Lockdep	5
2.4	RCU (Read-Copy-Update)	5
2.5	<code>igrab()</code> vs <code>ihold()</code> — Reference counting sugli inode	6
2.6	<code>iomap</code> e il completamento I/O asincrono	6
3	Primo tentativo: KASAN UAF in <code>fserror_worker</code>	6
3.1	Scelta iniziale del bug	6
3.2	Setup dell'ambiente di lavoro	7
3.3	Problemi di cross-compilazione	7
3.4	Riproduzione del bug	7
3.5	Analisi del crash KASAN	8
3.6	Analisi del codice sorgente	8
3.7	Patch tentativo	9
3.8	Motivazione del cambio di bug	9
4	Bug corrente: inconsistent lock state in <code>igrab</code>	10
4.1	Scelta del bug	10
4.2	Setup e asset	10
4.3	Il call trace lockdep	11
4.4	Lettura del warning e identificazione del problema	12
4.5	Lo scenario di deadlock	13
4.6	Localizzazione del problema	13
4.7	Riproduzione consistente: 20 boot separati	14
4.8	Perché il warning scatta solo una volta per boot	15
5	Compilazione del kernel <code>for-next</code> e verifica sul codice corrente	16
5.1	Repository di riferimento	16
5.2	Estrazione della configurazione syzbot	16
5.3	Compilazione	16
5.4	Avvio del kernel self-compiled e verifica della riproduzione	17
6	Interazione con il docente	18
6.1	Prima proposta e KASAN UAF (31 marzo – 3 aprile 2026)	18
6.2	Cambio di bug (13 aprile 2026)	19
6.3	La questione del reboot (13-14 aprile 2026)	19
6.4	Validazione del test 20/20 e indicazioni (14 aprile 2026)	19
6.5	Comunicazione del fix e meeting (15–16 aprile 2026)	20
6.6	Follow-up post-meeting: debug e analisi assembly (17 aprile 2026)	20
7	Progettazione del fix: <code>igrab()</code> → <code>ihold()</code>	21
8	Riconfigurazione del kernel e rebuild	22
9	Test di controllo: baseline senza fix	23
10	Applicazione del fix e verifica	23
11	Analisi della preconditione: debug e confronto <code>igrab/ihold</code>	24
11.1	Strumentazione di debug	24
11.2	Evidenza runtime: <code>log_debug.txt</code>	25

11.3	Confronto igrab vs ihold: espansione assembly	25
11.4	Conclusione sull'equivalenza	26
12	Generazione della patch e controlli di stile	26
12.1	Verifica con <code>checkpatch.pl</code>	28
12.2	Identificazione dei maintainer	28
13	Stato finale e prossimi passi	29
14	Riepilogo del lavoro svolto	29
15	Riferimenti	30

1 Introduzione

Il presente report documenta il lavoro svolto per il progetto del corso di Kernel Hacking, tenuto dal Prof. Paolo Valente presso l'Università di Modena e Reggio Emilia. L'obiettivo del corso è portare gli studenti a fornire contributi reali al kernel Linux, attraverso l'intero ciclo di sviluppo: dalla scelta del problema, alla riproduzione del bug, fino allo sviluppo e alla sottomissione di una patch.

Il lavoro si è articolato in due fasi. Una prima fase, descritta nel capitolo 3, ha riguardato il tentativo di riprodurre e correggere un bug di tipo *slab-use-after-free* nel sottosistema di segnalazione errori del filesystem (`fserror_worker`), rilevato da KASAN. Questo primo tentativo è stato abbandonato per ragioni metodologiche descritte nel seguito: pur avendo ottenuto una riproduzione iniziale, il bug si è rivelato di difficile riproduzione sistematica e una patch completa avrebbe richiesto un intervento profondo nel layer XFS. La seconda fase, che costituisce il cuore del progetto, si concentra su un bug distinto — un *inconsistent lock state* rilevato da lockdep nella funzione `igrab()`, chiamata da `fserror_report()` in contesto softirq — che si è rivelato riproducibile in modo altamente consistente (19 riproduzioni su 20 boot separati).

Il bug corrente è classificato come segue:

WARNING: inconsistent lock state in igrab
 Sottosistema: VFS / XFS / iomap — Commit: `cd7a5651db26`
 Tipo: Lockdep violation (`SOFTIRQ-ON-W` ↔ `IN-SOFTIRQ-W`)

Il report è strutturato come un *walkthrough* cronologico: descrive ogni fase del lavoro, i problemi incontrati, le soluzioni adottate e le conoscenze acquisite lungo il percorso. La scelta di mantenere documentata anche la prima fase abbandonata è deliberata: il processo decisionale che ha portato al cambio di bug è parte integrante del lavoro e ha valore metodologico.

2 Concetti di base

Prima di entrare nel merito dei bug, è necessario introdurre i concetti e gli strumenti fondamentali utilizzati durante il progetto.

2.1 Syzkaller e syzbot

Syzkaller è un fuzzer del kernel Linux sviluppato da Google. Genera sequenze casuali di system call per stressare il kernel e trovare bug. **Syzbot** è l'infrastruttura automatica che esegue syzkaller su diverse configurazioni del kernel, raccoglie i crash e li pubblica su un bug tracker pubblico.

Per ogni bug, syzbot fornisce:

- Un **C reproducer**: un programma C che riproduce la sequenza di operazioni che triggerano il bug.
- Il **kernel config** (`.config`): la configurazione esatta usata per compilare il kernel in cui il bug è stato rilevato.

- Gli **asset pre-compilati**: bzImage (kernel), vmlinux (kernel con simboli di debug), disco root, e immagini di filesystem usate dal reproducer.
- Lo **stack trace** del crash con informazioni dettagliate sulla causa.

2.2 KASAN (Kernel Address Sanitizer)

KASAN è uno strumento di *runtime* integrato nel kernel Linux che rileva accessi illegali alla memoria. Viene attivato tramite l'opzione `CONFIG_KASAN=y` nella configurazione del kernel e aggiunge *instrumentazione* a ogni accesso in memoria, verificando che l'indirizzo sia valido.

KASAN può rilevare diversi tipi di bug:

- **Use-after-free (UAF)**: accesso a memoria che è stata liberata con `kfree()` o restituita allo slab allocator. Era il tipo di bug del primo tentativo.
- **Out-of-bounds**: accesso oltre i limiti di un buffer allocato.
- **Use-after-scope**: accesso a variabili locali dopo che il loro scope è terminato.

Quando KASAN rileva un accesso illegale, stampa un report dettagliato che include indirizzo e dimensione dell'accesso, lo stack trace del punto di accesso e quello dell'allocazione/liberazione, e lo stato della *shadow memory* attorno all'indirizzo. Il valore `fb` nella shadow memory indica memoria appartenente a un oggetto slab già liberato.

2.3 Lockdep

Lockdep è il validatore di dipendenze di lock integrato nel kernel Linux. Attivato con `CONFIG_PROVE_LOCKING=y`, analizza a runtime ogni acquisizione di spinlock, mutex o read-write lock e verifica la coerenza dell'ordinamento. La sua forza è che rileva *potenziali* deadlock anche se non si manifestano effettivamente durante l'esecuzione: basta che la sequenza di acquisizioni osservata sia compatibile con uno scenario di deadlock per generare un warning.

Il controllo fondamentale rilevante per questo progetto riguarda la **coerenza softirq**. Se una *lock class* viene acquisita in un certo momento con le softirq abilitate (stato `SOFTIRQ-ON-W` per il write-mode) e poi la stessa lock class viene acquisita dall'interno di un softirq (stato `IN-SOFTIRQ-W`), lockdep segnala una possibile inconsistenza: se una softirq scattasse mentre un thread detiene il lock in process context, la softirq tenterebbe di acquisire lo stesso lock già detenuto, causando deadlock.

Lockdep non traccia singoli lock ma **classi di lock**, identificate staticamente dal punto di dichiarazione nel codice. Ogni classe ha un campo `usage_mask` che memorizza quali stati d'uso sono stati osservati. Un dettaglio rilevante per la riproduzione: **lockdep stampa il warning per una data inconsistenza una sola volta per boot**, perché una volta segnalato lo stato incoerente il bit viene settato in `usage_mask` e i controlli successivi terminano con un early return. Questo meccanismo influenza il protocollo di test (capitolo 4.6).

2.4 RCU (Read-Copy-Update)

RCU è un meccanismo di sincronizzazione del kernel Linux ottimizzato per scenari con molti lettori e pochi scrittori. Il concetto fondamentale è che la liberazione della memoria viene *differita* fino a quando tutti i lettori preesistenti hanno terminato la loro sezione critica.

Nel contesto del primo bug analizzato, XFS usa `call_rcu()` per liberare le strutture `xfs_inode` dallo slab allocator. Dopo che un inode viene marcato per la rimozione, la memoria effettiva

non viene restituita immediatamente: `kmem_cache_free()` viene invocato solo dopo che un *grace period* RCU è trascorso.

2.5 `igrab()` vs `ihold()` — Reference counting sugli inode

Il VFS Linux usa un contatore di riferimenti (`i_count`) per gestire il ciclo di vita degli inode:

- **`ihold(inode)`**: incrementa `i_count` incondizionatamente tramite un'operazione atomica. Richiede che il chiamante già posseda un riferimento valido (`i_count >= 1`). Non acquisisce `inode->i_lock`.
- **`igrab(inode)`**: versione «sicura» che controlla se l'inode è in fase di liberazione (`I_FREEING` o `I_WILL_FREE`). Se sì, restituisce `NULL` senza incrementare. Se no, incrementa `i_count` e restituisce il puntatore. **Per eseguire il check, acquisisce `inode->i_lock` tramite `spin_lock()`.**
- **`iput(inode)`**: decrementa `i_count`. Se raggiunge 0, avvia il processo di *eviction* dell'inode (che eventualmente porta alla liberazione della memoria).

La differenza tra `igrab()` e `ihold()` è centrale per il bug del capitolo 4: l'acquisizione di `i_lock` in `igrab()` rende la funzione insicura quando viene invocata da contesto `softirq`, mentre `ihold()` è safe da qualsiasi contesto.

2.6 `iomap` e il completamento I/O asincrono

`iomap` è un sottosistema relativamente moderno del VFS che fornisce una libreria unificata per il mapping tra blocchi di file e blocchi di device, utilizzata da XFS (e altri filesystem) per l'I/O. Quando un `bio` di lettura completa, il callback `iomap_read_end_io()` viene invocato dal layer a blocchi. Questa invocazione avviene in contesto **`softirq`**, perché `bio_endio()` è tipicamente chiamato da `blk_complete_reqs` durante il processamento degli interrupt di completamento I/O.

Questo è il punto critico: qualsiasi funzione chiamata lungo il path `bio_endio` → `iomap_read_end_io` → `iomap_finish_folio_read` → ... eredita il contesto `softirq` e deve quindi evitare operazioni non-atomiche (`sleep`, allocazioni bloccanti, acquisizione di lock non `softirq-safe`).

3 Primo tentativo: KASAN UAF in `fserror_worker`

Questa sezione documenta il primo bug affrontato e le ragioni che hanno portato a cambiare obiettivo. Il lavoro svolto non è andato perso: ha permesso di familiarizzare con l'infrastruttura `syzkaller`, di comprendere il file `fs/fserror.c` (lo stesso file su cui si concentra anche il bug corrente) e di sviluppare il workflow di riproduzione in QEMU.

3.1 Scelta iniziale del bug

La scelta è stata guidata da criteri di sicurezza (il mio percorso di studi è sulla sicurezza informatica), disponibilità di un C reproducer, e fattibilità della riproduzione in ambiente virtualizzato. Il bug selezionato è stato:

Titolo: KASAN: slab-use-after-free in `fserror_worker`

URL: <https://syzkaller.appspot.com/bug?extid=fbf6ff30de890ff32ec5>

Sottosistema: VFS / XFS
Head commit: dc855b77719f
Compilatore: Debian clang 21.1.8
Architettura: x86_64 (QEMU Q35)

Il bug si manifesta quando `fserror_worker()` — un worker del workqueue `events` — chiama `iput()` su un inode XFS già liberato tramite un callback RCU. KASAN rileva la lettura di 4 byte (il campo `i_count`, un `refcount_t`) da una regione slab marcata come libera.

3.2 Setup dell'ambiente di lavoro

Il setup di sviluppo è il seguente, e resta valido per l'intero progetto:

Host: macOS su Apple Silicon (ARM64)
VM: Debian GNU/Linux su UTM (Apple Virtualization Framework) — ARM64
Emulazione: QEMU `qemu-system-x86_64` dentro la VM — emulazione x86_64 su ARM64
RAM VM: 8 GB
Target: kernel Linux x86_64

Sulla VM Debian ARM64 sono stati installati gli strumenti necessari:

```
sudo apt install qemu-system-x86 gcc-x86-64-linux-gnu \
    build-essential flex bison libelf-dev libssl-dev bc
```

3.3 Problemi di cross-compilazione

Il primo ostacolo è stato la compilazione del kernel x86_64 sulla VM ARM64: `objtool`, che valida gli stack frame per ORC unwinding, richiedeva circa 13 GB di RAM per processare `vmlinux.o` con KASAN abilitato, a fronte degli 8 GB disponibili sulla VM. Anche con swap aggiuntivo il sistema andava in *thrashing* severo, rendendo il processo impraticabile.

La soluzione è stata utilizzare i binari pre-compilati che syzbot pubblica per ogni bug: `bzImage`, `disk.raw` (root filesystem Debian) e l'immagine del filesystem XFS corrotta dal fuzzer. Questo ha permesso di aggirare completamente il problema di compilazione per la fase di riproduzione iniziale.

3.4 Riproduzione del bug

Dopo aver scaricato e decompresso gli asset, il reproducer C è stato cross-compilato per x86_64:

```
x86_64-linux-gnu-gcc -o repro_x86 repro.c -static -lpthread
```

QEMU è stato avviato con kernel, disco root e immagine XFS:

```
qemu-system-x86_64 \
    -m 2048 -smp 2 \
```

```
-kernel bzImage \
-append "console=ttyS0 root=/dev/sda1 earlyprintk=serial \
       net.ifnames=0 nokaslr panic=1" \
-drive file=disk.raw,format=raw,if=ide,index=0 \
-drive file=mount_1,format=raw,if=ide,index=1 \
-net nic,model=e1000 \
-net user,hostfwd=tcp::10022-:22 \
-nographic -no-reboot
```

Il reproducer è stato eseguito in istanze parallele per aumentare la probabilità di colpire la race window. Al secondo run, il crash KASAN è stato riprodotto:

```
BUG: KASAN: slab-use-after-free in iput+0x3a7/0xe80
Read of size 4 at addr ffff888022e202b8 by task kworker/0:0/6055

CPU: 0 UID: 0 PID: 6055 Comm: kworker/0:0 Not tainted
Workqueue: events fserror_worker
Call Trace:
<TASK>
dump_stack_lvl+0xe8/0x150
print_report+0xba/0x230
kasan_report+0x117/0x150
iput+0x3a7/0xe80
fserror_worker+0x230/0x350
process_scheduled_works+0xaec/0x17a0
worker_thread+0xda6/0x1360
kthread+0x388/0x470
ret_from_fork+0x51e/0xb90
</TASK>
```

3.5 Analisi del crash KASAN

Il crash report indica una lettura di 4 byte (il campo `i_count` della struttura `inode`) all'offset 696 all'interno di un oggetto `xfs_inode` di 1784 byte della cache slab omonima. L'indirizzo accede a memoria con shadow byte `fb`, che segnala una regione restituita allo slab allocator.

Il task responsabile è `kworker/0:0` (PID 6055), e il workqueue è `events`, worker function `fserror_worker`. Lo stack trace dell'allocazione risale a `xfs_inode_alloc` → `xfs_iget` → `xfs_lookup` → ... → `newfstatat`, mentre la liberazione proviene da un callback RCU eseguito durante un `softirq` (`rcu_core` → `kmem_cache_free`). La «Last potentially related work creation» mostra che `call_rcu` è stata invocata a un offset diverso di `xfs_iget`, indicando un percorso di errore interno a `xfs_iget` stesso.

3.6 Analisi del codice sorgente

Il file centrale è `fs/fserror.c`, che implementa l'infrastruttura di segnalazione errori del filesystem. Le due funzioni chiave sono:

`fserror_report()`: chiamata dai filesystem quando rilevano un errore. Alloca un `fserror_event`, prende un riferimento sull'inode con `igrab()`, e schedula il worker tramite `schedule_work()`.

`fserror_worker()`: eseguito nel `workqueue events`. Invoca il callback `sb->s_op->report_error()` e la notifica `fsnotify`, poi rilascia il riferimento all'inode con `iput(event->inode)` e libera l'evento. È qui che scatta il crash.

La sequenza ricostruita del bug:

1. `xfs_iget()` alloca un `xfs_inode` e tenta di leggere l'inode dal disco.
2. Rileva metadati corrotti e chiama `xfs_inode_mark_sick()` → `fserror_report()`.
3. `fserror_report()` chiama `igrab(inode)`: `I_FREEING` non è impostato, quindi `i_count` passa da 1 a 2 e l'evento viene schedulato.
4. `xfs_iget()` ritorna errore e libera l'inode **direttamente** con `call_rcu(xfs_inode_free_callback)`, bypassando completamente il reference counting VFS.
5. Il grace period RCU termina, il callback esegue `kmem_cache_free()`, la slab viene restituita e KASAN avvelena la regione.
6. Il workqueue schedula `fserror_worker()`, che accede a `event->inode` e chiama `iput()` → accesso a memoria avvelenata → crash.

La root cause è quindi che `igrab()` non protegge da una liberazione diretta che bypassa il VFS. Il contratto di `igrab()` presuppone che *tutte le liberazioni* passino per il VFS (ovvero `iput()` → `evict()` → `call_rcu()`), ma nel percorso di errore di `xfs_iget()` questo contratto viene violato.

3.7 Patch tentativa

Su suggerimento del Prof. Valente («potresti inserire un controllo nel codice che passa l'inode al worker, e segnalare errore se l'inode ha il counter a 0»), la fix è stata strutturata in due commit: un commit diagnostico che aggiunge un `WARN_ON_ONCE` se `i_count == 0` al momento della chiamata, e un commit di fix che trasforma il warning in un guard, saltando a `lost_event` per prevenire il use-after-free. I due file `0001-fserror.patch` e `0002-fserror.patch` presenti nella cartella `Progetto/` sono le patch generate con `git format-patch`.

La patch ha un limite noto: copre il caso in cui `i_count == 0` al momento della chiamata a `fserror_report()`, ma esiste un caso residuo in cui `i_count > 0` al check e `igrab()` ha successo, ma poi XFS libera la slab direttamente. Una fix completa richiederebbe un intervento nel layer XFS (ad esempio impedire che `xfs_iget()` chiami `xfs_inode_mark_sick()` su inode non ancora inseriti nella inode cache VFS).

3.8 Motivazione del cambio di bug

Dopo la riproduzione iniziale, sono emerse difficoltà nel riprodurre il bug in modo sistematico. Essendo una race condition sensibile al timing, la finestra di colpo era molto stretta: la prima riproduzione è stata poco più che un colpo di fortuna, e i tentativi successivi non hanno prodotto il crash in modo affidabile. L'emulazione TCG (senza KVM) di x86_64 su ARM64 modifica in modo significativo i pattern di scheduling e questo rende la race ancora più elusiva.

Dopo consultazione con il prof., ho deciso di cambiare bug per le seguenti ragioni:

1. Una riproduzione saltuaria rende difficile verificare l'efficacia della patch: senza una riproduzione consistente non è possibile distinguere tra «la patch funziona» e «la race semplicemente non è scattata in questo run».
2. La fix completa richiederebbe un intervento nel layer XFS che va oltre il perimetro ragionevole di un progetto di corso.

3. Il tempo restante per completare il progetto era limitato e concentrarsi su un bug più trattabile appariva la scelta metodologicamente più solida.

Il lavoro svolto sul primo bug rimane utile: la familiarità acquisita con il file `fs/fserror.c`, con il workflow `syzkaller` + `QEMU` e con `KASAN` si è rivelata direttamente applicabile al bug corrente, che condivide lo stesso file sorgente e lo stesso tipo di infrastruttura.

4 Bug corrente: inconsistent lock state in igrab

4.1 Scelta del bug

Il bug selezionato per la seconda fase è:

Titolo: WARNING: inconsistent lock state in igrab
URL: <https://syzkaller.appspot.com/bug?extid=5eb0d61dfb76ca12670c>
Sottosistema: VFS / XFS / iomap / fserror
Head commit: cd7a5651db263b5384aef1950898e5e889425134
Tipo: Lockdep violation — inconsistent {SOFTIRQ-ON-W} → {IN-SOFTIRQ-W} usage
Architettura: x86_64 (QEMU Standard PC i440FX)

La scelta è stata guidata da un criterio aggiuntivo rispetto al primo tentativo: la **consistenza della riproduzione**. Prima di impegnarsi sul fix, ho eseguito una batteria di prove preliminari ottenendo 3 riproduzioni su 4 boot, percentuale sufficiente a giustificare un protocollo di test più esteso (capitolo 4.5).

4.2 Setup e asset

Gli asset di syzbot per questo bug sono stati scaricati in `~/repro/`:

File	Dimensione	Descrizione
<code>bzImage-cd7a5651</code>	100 MB	Kernel x86_64 pre-compilato con lockdep/KASAN
<code>disk-cd7a5651.raw</code>	1 GB	Root filesystem con tabella MBR, Linux root su <code>/dev/sda1</code>
<code>repro.c</code>	300 KB	C reproducer generato da syzkaller

Il reproducer è stato cross-compilato staticamente per x86_64:

```
x86_64-linux-gnu-gcc -o ~/repro/repro ~/repro/repro.c -lpthread -static
```

Il layout del disco è stato verificato con `fdisk -l` / parser MBR, confermando la partizione Linux a partire da LBA 310. Il kernel viene avviato con `root=/dev/sda1`.

Il comando QEMU utilizzato:

```
qemu-system-x86_64 \
  -kernel ~/repro/bzImage-cd7a5651 \
```

```
-drive file=~/.repro/disk-cd7a5651.raw,format=raw,if=ide \
-append "root=/dev/sda1 console=ttyS0 panic_on_warn=1 \
        panic=1 earlyprintk=serial" \
-virtfs local,path=/home/luffy/repro,mount_tag=host0,\
security_model=mapped,id=host0 \
-m 2G -smp 2 -nographic
```

La differenza principale rispetto al primo tentativo è l'opzione `panic_on_warn=1` al posto di `kasan.fault=panic`: il bug corrente è un *lockdep warning*, non una violazione KASAN. Con `panic_on_warn=1`, qualsiasi `WARN_ON` (incluso quello emesso da `lockdep`) causa un kernel panic immediato, rendendo la cattura del warning affidabile.

La directory `~/repro/` dell'host è esportata nella VM tramite `virtfs` (9p), così il riproduttore può essere eseguito da dentro QEMU senza copiarlo nel disco:

```
mkdir -p /mnt/host
mount -t 9p -o trans=virtio host0 /mnt/host
/mnt/host/repro
```

4.3 Il call trace lockdep

All'esecuzione del riproduttore, dopo qualche iterazione di `mount/unmount XFS` e di `I/O errors`, `lockdep` emette il seguente warning (estratto significativo del run 1):

```
=====
WARNING: inconsistent lock state
syzkaller #0 Not tainted
-----
inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage.
ksoftirqd/0/15 [HC0[0]:SC1[1]:HE1:SE0] takes:
(&sb->s_type->i_lock_key#34){+?.}-{3:3}, at: igrab+0x2a/0x230

{SOFTIRQ-ON-W} state was registered at:
  _raw_spin_lock+0x2e/0x40
  unlock_new_inode+0x7a/0x330
  xfs_iget+0x1196/0x30f0
  xfs_mountfs+0x108c/0x20b0
  xfs_fs_fill_super+0x153f/0x1f20
  ...

Possible unsafe locking scenario:
    CPU0
    ----
    lock(&sb->s_type->i_lock_key#34);
    <Interrupt>
    lock(&sb->s_type->i_lock_key#34);
    *** DEADLOCK ***

Call Trace:
<TASK>
  igrab+0x2a/0x230
```

```

fserror_report+0x3fd/0x750
iomap_finish_folio_read+0x168/0x2c0
iomap_read_end_io+0x11a/0x430
bio_endio+0x755/0x8b0
blk_update_request+0x741/0x1330
blk_mq_end_request+0x5b/0x420
lo_complete_rq+0x1d4/0x2a0
blk_complete_reqs+0xb1/0xf0
handle_softirqs+0x1ea/0x910
run_ksoftirqd+0x38/0x60
smpboot_thread_fn+0x3d3/0xaa0
...
</TASK>

```

4.4 Lettura del warning e identificazione del problema

Il warning va letto in tre parti.

Prima parte — chi sta acquisendo il lock ora:

```

inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage.
ksoftirqd/0/15 [HC0[0]:SC1[1]:HE1:SE0] takes:
(&sb->s_type->i_lock_key#34){+?.}- {3:3}, at: igrab+0x2a/0x230

```

Un thread `ksoftirqd` sta acquisendo lo spinlock di lock class `i_lock_key#34` dentro `igrab()` (offset `+0x2a`, cioè la `spin_lock(&inode->i_lock)` nel prologo della funzione). Il flag `SC1` in `HC0[0]:SC1[1]:HE1:SE0` indica che il thread è **dentro un softirq** (SoftIRQ Count = 1). La lock class è marcata come `i_lock_key#34`: il `#34` è l'identificatore univoco associato al tipo di lock class per gli inode XFS (lockdep assegna un numero progressivo per ogni filesystem), e il `{+?.}- {3:3}` codifica gli stati già osservati per questa classe.

Seconda parte — dove era stato registrato lo stato `SOFTIRQ-ON-W`:

```

{SOFTIRQ-ON-W} state was registered at:
_raw_spin_lock+0x2e/0x40
unlock_new_inode+0x7a/0x330
xfs_iget+0x1196/0x30f0
xfs_mountfs+0x108c/0x20b0
xfs_fs_fill_super+0x153f/0x1f20
...
__x64_sys_mount+0x293/0x310

```

Questo stack trace è di un *evento passato*: in un momento precedente della vita del kernel, lo stesso `i_lock_key#34` era stato acquisito durante una `mount()` di XFS, nel percorso `path_mount` → `xfs_fs_fill_super` → `xfs_mountfs` → `xfs_iget` → `unlock_new_inode`. In quel momento, le softirq erano abilitate (process context), da cui lo stato `SOFTIRQ-ON-W`.

Terza parte — il call trace attuale:

```

igrab+0x2a/0x230
fserror_report+0x3fd/0x750
iomap_finish_folio_read+0x168/0x2c0
iomap_read_end_io+0x11a/0x430
bio_endio+0x755/0x8b0
blk_update_request+0x741/0x1330
blk_mq_end_request+0x5b/0x420
lo_complete_rq+0x1d4/0x2a0
blk_complete_reqs+0xb1/0xf0
handle_softirqs+0x1ea/0x910
run_ksoftirqd+0x38/0x60

```

Si legge dal basso verso l'alto:

- Il thread `ksoftirqd` sta eseguendo la BLOCK `softirq` (`blk_complete_reqs`).
- Il completamento di una request del loop device (`lo_complete_rq`) chiama `blk_mq_end_request` → `blk_update_request` → `bio_endio`.
- `bio_endio` invoca il callback `endio` del bio di lettura: `iomap_read_end_io` → `iomap_finish_folio_read`.
- `iomap_finish_folio_read` chiama `fserror_report()` per notificare l'errore di I/O.
- `fserror_report()` chiama `igrab(inode)` all'offset `+0x3fd` per prendere un riferimento sull'`inode`.
- `igrab()` acquisisce `inode->i_lock` via `spin_lock()`.

4.5 Lo scenario di deadlock

La **Possible unsafe locking scenario** riportata da `lockdep` è:

```

CPU0
----
lock(&sb->s_type->i_lock_key#34);  // process context, softirq abilitate
<Interrupt>
lock(&sb->s_type->i_lock_key#34); // softirq scatta, re-acquire: DEADLOCK

```

Cioè: un thread in process context (tipicamente la path di mount) detiene `inode->i_lock`. Una `softirq` scatta su questa CPU. La `softirq` tenta di acquisire **lo stesso** `inode->i_lock` tramite `igrab()`. Lo `spinlock` è già detenuto dal thread interrotto, che però non potrà mai rilasciarlo perché è stato interrotto: `deadlock self-inflicted`.

`Lockdep` non ha osservato il deadlock reale — ha osservato una sequenza di acquisizioni che lo rende **possibile** sulla stessa lock class. Questo basta a generare il warning.

4.6 Localizzazione del problema

La funzione colpevole è `fserror_report()` nel file `fs/fserror.c`. Il codice problematico (riga 159 nella `for-next` attuale) è:

```

if (inode) {
    event->inode = igrab(inode);
    if (!event->inode)

```

```

    goto lost_event;
}

```

Il problema non è né in `iomap_finish_folio_read` (che fa correttamente il suo lavoro in `softirq`) né in `igrab()` stesso (che è `safe` per essere chiamata in `process context`): il problema è che `fserror_report()` chiama `igrab()` — una funzione che acquisisce uno `spinlock non-softirq-safe` — anche quando invocata in contesto `softirq`.

4.7 Riproduzione consistente: 20 boot separati

Su indicazione del Prof. Valente, è stato effettuato un test sistematico: 20 boot di QEMU separati, un reboot completo tra ogni prova, per misurare la percentuale di riproduzione.

Protocollo:

1. Avvio di QEMU con il comando descritto al capitolo 4.2.
2. Dentro la VM: `mkdir -p /mnt/host && mount -t 9p -o trans=virtio host0 /mnt/host && /mnt/host/repro.`
3. Osservazione dell'output seriale in attesa del blocco `WARNING: inconsistent lock state` (timeout 60 secondi).
4. Uscita da QEMU con `Ctrl+A X`.
5. Registrazione dell'esito.

Risultati:

Boot	Stato	Inizio (s)	Warning (s)	Note
1	✓	—	277	—
2	✓	—	256	—
3	✓	—	241	—
4	✓	—	—	dato mancante
5	✗	244.53	245.49	FS shutdown prima del warning
6	✓	—	251	—
7	✓	—	442	—
8	✓	407	413	—
9	✓	408	428	—
10	✓	608	634	—
11	✓	501	505	—
12	✓	375	385	—
13	✓	606	607	—
14	✓	580	592	—
15	✓	274	275	—
16	✓	248	253	—
17	✓	435	441	—
18	✓	533	539	—
19	✓	431.04	431.76	—
20	✓	253	258	—

Esito: 19 riproduzioni su 20, pari al **95%**. Il singolo fallimento (boot 5) non è stato una race window mancata ma una divergenza del repro: il filesystem XFS è andato in shutdown per log I/O error prima che il percorso `iomap_finish_folio_read` → `fserror_report` → `igrab` potesse essere percorso. In tutti gli altri boot il warning è scattato tra 1 e 26 secondi dall'inizio del primo `executing program` del repro.

4.8 Perché il warning scatta solo una volta per boot

Durante il test sistematico è emerso un comportamento che inizialmente pareva una race condition aggiuntiva: il warning scatta una sola volta, poi le iterazioni successive del repro continuano a triggerare lo stesso percorso di codice senza produrre alcun output di lockdep. Solo il reboot del kernel ripristina la possibilità di vedere di nuovo il warning.

La causa non è una race: è una proprietà **by-design** del meccanismo interno di lockdep.

Lockdep non traccia singoli lock ma **lock class**. Ogni classe ha un campo `usage_mask` — una bitmask che registra gli stati d'uso osservati (`SOFTIRQ-ON-W`, `IN-SOFTIRQ-W`, `HARDIRQ-ON-W`, ecc.). Il controllo di coerenza in `mark_lock()` (file `kernel/locking/lockdep.c`) è eseguito **solo** quando si sta per aggiungere un bit nuovo a `usage_mask`:

```
if (likely(hlock->class->usage_mask & new_mask))
    return 1; // stato già noto, skip del check
```

La sequenza concreta nella riproduzione è:

1. **Boot:** tutti i `usage_mask` sono a zero.
2. **Mount XFS:** `xfs_iget` → `unlock_new_inode` acquisisce un `inode->i_lock` in process context con softirq abilitate. Lockdep aggiunge `SOFTIRQ-ON-W` a `i_lock_key#34->usage_mask`. Solo un bit settato, nessuna inconsistenza.
3. **Primo bio read che fallisce:** `iomap_finish_folio_read` → `fserror_report` → `igrab` acquisisce `i_lock` in softirq. Lockdep tenta di aggiungere `IN-SOFTIRQ-W`: il bit non è ancora settato, il check completo viene eseguito, rileva l'incompatibilità con `SOFTIRQ-ON-W` già presente, stampa il warning, poi setta il bit.
4. **Ogni chiamata successiva a igrab in softirq:** il bit `IN-SOFTIRQ-W` è già in `usage_mask`, l'early return scatta, nessun check, nessun warning.

Al reboot, la memoria dei lock class viene reinizializzata a zero e il ciclo ricomincia.

Va sottolineato: non si tratta di una soppressione esplicita, e non è legato ad alcun meccanismo di «deadlock handling» del kernel. È semplicemente che, una volta che entrambi gli stati sono stati osservati, non esistono più bit nuovi da aggiungere — quindi il codice di verifica non viene più eseguito. La conseguenza pratica per il protocollo di test è che **ogni test deve essere un boot fresco di QEMU**, come di fatto è stato fatto. Una volta accettato questo vincolo il costo non è elevato: ogni boot richiede circa 30-45 secondi.

5 Compilazione del kernel **for-next** e verifica sul codice corrente

La riproduzione al capitolo 4 è avvenuta sul **bzImage** pre-compilato di syzbot, corrispondente al commit `cd7a5651db26`. Prima di scrivere qualsiasi patch è necessario verificare che il bug sia ancora presente nella versione corrente dell'albero XFS, compilando personalmente il kernel e confermando la riproduzione. Questo garantisce che il fix sarà sviluppato contro codice attuale e non contro un commit ormai superato.

5.1 Repository di riferimento

Il repository utilizzato è l'albero di sviluppo XFS:

```
~/xfs-linux/
branch: for-next
HEAD:    2ffc6900d5c3a7cd59becda2aa67581d9bd3858e
        ("xfs: fix number of GC bvecs", 6 apr 2026)
merge:   origin/xfs-7.1-merge, origin/for-next
```

Il HEAD è successivo al commit buggy `cd7a5651db26` (che appartiene alla stessa linea evolutiva), quindi è il candidato naturale per verificare se il bug persiste nel codice corrente.

Una verifica diretta con **grep** conferma che la chiamata incriminata è ancora presente:

```
luffy@evil-corp:~/xfs-linux$ grep -n "igrab" ~/xfs-linux/fs/fserror.c
159:         event->inode = igrab(inode);
```

Il codice sorgente non è stato modificato dopo la segnalazione di syzbot: il bug è ancora aperto nell'albero di sviluppo.

5.2 Estrazione della configurazione syzbot

Per garantire che il kernel self-compiled abbia lockdep, KASAN e tutti gli altri sanitizer abilitati, la configurazione è stata estratta direttamente dal **bzImage** di syzbot:

```
cd ~/xfs-linux
scripts/extract-ikconfig ~/repro/bzImage-cd7a5651 > .config
make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- olddefconfig
```

`olddefconfig` risolve le nuove opzioni Kconfig introdotte tra `cd7a5651` e `2ffc6900d5c3` accettando i default senza richiedere interazione.

5.3 Compilazione

Sul primo tentativo la compilazione è stata interrotta da un SIGKILL (exit code 137) durante la fase di linking di `vmlinux.o`:


```
LD      vmlinux.o
Killed
make[2]: *** [scripts/Makefile.vmlinux_o:76: vmlinux.o] Error 137
```

Causa: l'OOM killer della VM — il linker richiede più RAM di quella fisicamente disponibile (8 GB) quando si usa `-j$(nproc)` su una configurazione con `lockdep` + `KASAN` abilitati.

La soluzione è stata aggiungere spazio di swap e ridurre la parallelizzazione del build:

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- -j2 bzImage
```

Con `-j2` il linking è passato, impiegando alcuni minuti in più ma senza esaurire la memoria. Il build si è concluso correttamente:

```
Kernel: arch/x86/boot/bzImage is ready (#1)
```

La stringa di versione del kernel risultante è:

```
7.0.0-rc1-g2ffc6900d5c3
```

5.4 Avvio del kernel self-compiled e verifica della riproduzione

Il kernel compilato è stato avviato in QEMU, sostituendo il `bzImage` di `syzbot` ma mantenendo invariati disco e reproducer:

```
qemu-system-x86_64 \
  -kernel ~/xfs-linux/arch/x86/boot/bzImage \
  -drive file=~/repro/disk-cd7a5651.raw,format=raw,if=ide \
  -append "root=/dev/sda1 console=ttyS0 panic_on_warn=1 \
          panic=1 earlyprintk=serial" \
  -virtfs local,path=/home/luffy/repro,mount_tag=host0,\
  security_model=mapped,id=host0 \
  -m 2G -smp 2 -nographic
```

All'esecuzione del reproducer, il warning è apparso al primo tentativo, a 303 secondi di uptime. Estratto essenziale dell'output:

```
[ 303.457715][    C1] =====
[ 303.457862][    C1] WARNING: inconsistent lock state
[ 303.458103][    C1] 7.0.0-rc1-g2ffc6900d5c3 #1 Not tainted
[ 303.458298][    C1] -----
[ 303.458453][    C1] inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage.
[ 303.458673][    C1] ksoftirqd/1/23 ...
[ 303.459011][    C1] ... i_lock_key#34 ... at: igrab+0x2a/0x230
```

```
[ 303.462117][ C1] {SOFTIRQ-ON-W} state was registered at:
[ 303.462409][ C1]   _raw_spin_lock+0x2e/0x40
[ 303.462902][ C1]   unlock_new_inode+0x7a/0x330
[ 303.463232][ C1]   xfs_iget+0x11d5/0x32c0
[ 303.463594][ C1]   xfs_mountfs+0x1086/0x20a0
[ 303.463915][ C1]   xfs_fs_fill_super+0x1541/0x1f80
...
Call Trace:
  igrab+0x2a/0x230
  fserror_report+0x3fd/0x750
  iomap_finish_folio_read+0x168/0x2c0
  iomap_read_end_io+0x11a/0x430
  bio_endio+0x7a3/0x910
  blk_update_request+0x741/0x13a0
  blk_mq_end_request+0x5b/0x420
  lo_complete_rq+0x1d4/0x2a0
  blk_complete_reqs+0xb1/0xf0
  handle_softirqs+0x1eb/0x9e0
  run_ksoftirqd+0x38/0x60
```

Il risultato conferma ciò che era atteso: il bug è presente nel kernel compilato dal sorgente `for-next` corrente, con lo **stesso esatto call trace** osservato sul kernel di syzbot. I piccoli scostamenti negli offset (`xfs_iget+0x11d5` invece di `+0x1196`, `bio_endio+0x7a3` invece di `+0x755`) sono dovuti alle modifiche intervenute nel codice circostante tra `cd7a5651db26` e `2ffc6900d5c3` — non alterano la sostanza del problema, che resta l’acquisizione di `inode->i_lock` in `igrab()` dal path `softirq`.

Questo passo chiude la fase di verifica: ho piena libertà di modificare `fs/fserror.c`, ricompilare incrementalmente (solo il file modificato + il linking, nell’ordine di pochi minuti anziché la ventina di minuti del build iniziale) e misurare l’effetto sulle prossime iterazioni.

6 Interazione con il docente

Questa sezione riporta i passaggi chiave dello scambio con il Prof. Valente, rilevanti per le decisioni prese nel corso del progetto.

6.1 Prima proposta e KASAN UAF (31 marzo – 3 aprile 2026)

Francesco, 31 mar 2026

[...] la mia idea era quella di patchare un bug preso dalla lista della piattaforma Syzkaller. Il bug individuato sarebbe questo: <https://syzkaller.appspot.com/bug?extid=fbf6ff30de890ff32ec5>.

Prof. Valente, 1 apr 2026

Buongiorno, va benissimo. Comincio con la prima domanda di rito: riesci a riprodurlo?

Dopo i primi problemi di cross-compilazione, la soluzione degli asset pre-compilati di syzbot ha permesso la prima riproduzione il 1 aprile alle 17:41. Il prof ha confermato la plausibilità dell'analisi iniziale sulla race condition, suggerendo l'approccio diagnostico di inserire un controllo su `i_count == 0` prima di schedulare il worker.

6.2 Cambio di bug (13 aprile 2026)

Francesco, 13 apr, 12:58

Si, però non son più riuscito. Avevo sempre eseguito con qemu, anche la prima volta. Magari ci si riesce, però vorrei evitare di avere una riproduzione saltuaria, che mi renderebbe il fix anche più complicato e dispendioso in termini di tempo. Ora ho trovato un altro bug sempre in XFS in cui avviene un warning di inconsistent lock state, e questo son riuscito, sempre con qemu, a riprodurlo per 3/4 volte di fila.

Prof. Valente, 13 apr, 13:01

Bravissimo, giusto. [...] Ok, fai 20 prove, aspettando 10-20 secondi tra ogni prova, e controlla con che percentuale ti si ripete. Se si riproduce in modo consistente, allora è tutto tuo!

6.3 La questione del reboot (13-14 aprile 2026)

Francesco, 13 apr, 13:07

[...] Sto riscontrando la riproduzione del bug solamente al reboot. Una volta ottenuto il warning per riottenerlo devo fare reboot del sistema. Non so se è per un qualche meccanismo del kernel che gestisce i deadlock?

Prof. Valente, 13 apr, 13:12

Quasi sicuramente si tratta dell'ennesima corsa critica, e per questioni fortuite la corsa ti accade solo dopo un reboot. [...] A te la decisione della fattibilità.

Il comportamento si è poi rivelato non essere una race condition aggiuntiva, ma una proprietà interna di lockdep (meccanismo `usage_mask` descritto al capitolo 4.6).

6.4 Validazione del test 20/20 e indicazioni (14 aprile 2026)

Francesco, 14 apr, 11:27

Ho terminato il test da lei proposto. Ho eseguito 20 riproduzioni, separate da un reboot ogni volta, ed ho ottenuto 19 riproduzioni su 20. [tabella risultati]

Prof. Valente, 14 apr, 13:21

Ok allora direi che hai accettato questa sfida. Comunque, per questo bug ci sta che non ti serva una riproduzione continua, perché direi che dovrai principalmente riflettere sullo scenario che ti propone lockdep e capire dove sta il problema e come si potrebbe risolvere. Se hai bisogno di aiuto chiedi pure, e se necessario commuteremo su un meeting.

Con 19/20 il bug è stato accettato e il focus del progetto si è spostato sull'analisi dello scenario lockdep e sulla progettazione del fix.

6.5 Comunicazione del fix e meeting (15–16 aprile 2026)

Francesco, 15 apr, 13:16

Buongiorno Prof, credo di essere riuscito a fixare il warning. Il problema risiede nella funzione `fserror_report()` in `fs/fserror.c`, esattamente dove il call trace di lockdep indica: alla chiamata a `igrab(inode)`. [segue descrizione tecnica del lock state `SOFTIRQ-ON-W ↔ IN-SOFTIRQ-W`, dello scenario di deadlock e della sostituzione con `ihold()`, con la giustificazione di correttezza basata sul riferimento detenuto dal page cache sulla folio sotto I/O]. Ho verificato la riproduzione del warning sul kernel non patchato e la sua assenza sul kernel patchato.

Prof. Valente, 15 apr, 18:55

Molto bene. Però ho bisogno che tu mi faccia vedere con calma tutti i frammenti di codice coinvolti. Al momento sono disponibile anche domani, sia la mattina che la sera.

Francesco, 16 apr, 15:12 (dopo scambio sull'orario)

Sisi, alle 16:30 è perfetto, poi magari posso anche rimanere che sarei curioso di sentire anche la sua idea.

Il meeting è stato tenuto giovedì 16 aprile alle 16:30 su Google Meet, in sessione condivisa con un altro studente (Adriano). Il Prof. Valente ha richiesto di vedere con calma i frammenti di codice coinvolti e, in particolare, di dimostrare più rigorosamente che `igrab ≡ ihold` nel contesto specifico.

6.6 Follow-up post-meeting: debug e analisi assembly (17 aprile 2026)

In risposta alle richieste emerse dal meeting, il giorno successivo è stata effettuata un'analisi in due parti — strumentazione di debug e confronto assembly — documentata nel capitolo 11.

Francesco, 17 apr 2026

Ho inserito vari debug in varie funzioni e le allego l'intero log, anche se sono 3k righe. Ho eseguito delle statistiche, per darle una panoramica, le allego anche quelle. In più ho analizzato tutte le chiamate di `igrab` e `ihold` fino al livello assembly, per capire l'effettivo funzionamento, e le ho scritto un piccolo report che può leggere.

Al netto dell'analisi, non ho trovato sostanziali differenze. `igrab()` acquisisce il lock per controllare se l'inode sta per essere liberato, mentre `ihold` non se ne preoccupa. La domanda era: «Quando viene chiamato `fserror_report`, si può essere sicuri che il reference counter sia ≥ 1 ?». Con il debug è venuto fuori che il reference counter è sempre ≥ 1 quando viene chiamata `fserror_report`, e le flag `I_FREEING` e `I_WILL_FREE` sono sempre a 0, rendendo il controllo di `igrab()` superfluo.

I tre allegati inviati al docente sono `log_debug.txt` (log completo con i messaggi di debug), `stats_log.txt` (analisi statistica delle 22 chiamate tracciate) e `igrab-vs-ihold.pdf` (espansione assembly delle due funzioni).

7 Progettazione del fix: `igrab()` $\rightarrow$ `ihold()`

Con il bug confermato riproducibile sia sul `bzImage` di `syzbot` sia sul kernel `for-next` self-compiled, il passo successivo è la progettazione della correzione. La domanda guida è semplice: cosa serve realmente in quel punto del codice? Il contesto è il seguente.

`fserror_report()` riceve un inode e ne deve trattenere un riferimento per tutta la durata del work item asincrono, che eseguirà la notifica `fsnotify` e chiamerà `iput()` al termine. Lo scopo di `igrab()` nel codice originale non è prendere uno spinlock — quello è un effetto collaterale del check `I_FREEING` — ma **incrementare `i_count` in modo sicuro rispetto alla possibilità che l'inode stia per essere liberato.**

Il punto chiave è che questa protezione è ridondante nel path che triggera il bug. `fserror_report()` viene chiamata lungo `bio_endio` $\rightarrow$ `iomap_read_end_io` $\rightarrow$ `iomap_finish_folio_read`, e in questo percorso l'inode è già protetto da un riferimento che il page cache mantiene sul folio sotto I/O. Un inode con una folio in page cache con I/O in flight non può essere in stato `I_FREEING`: il VFS non inizia l'eviction di un inode che ha pagine dirty o con I/O pendente. Il check di `igrab()` è quindi superfluo e, peggio, introduce un'acquisizione di spinlock in un contesto che non può permetterselo.

La soluzione è sostituire `igrab()` con `ihold()`:

Aspetto	<code>ihold()</code>
Operazione	incremento atomico di <code>i_count</code> (<code>atomic_inc</code>)
Lock acquisiti	nessuno
Precondizione richiesta	il chiamante deve già detenere un riferimento
Costo	una singola istruzione atomica
Safety in softirq	sì, sempre

La precondizione «il chiamante deve già detenere un riferimento» è quella soddisfatta dal path di `iomap_finish_folio_read`. Il cambio al chiamante non è necessario: la semantica esterna di `fserror_report()` resta identica, cambia solo il meccanismo interno di acquisizione del riferimento.

La modifica al codice coinvolge tre righe effettive in `fs/fserror.c` (la sostituzione di `igrab` con `ihold` in coppia con l'assegnamento separato, e la rimozione dell'error path `lost_event` che diventa irraggiungibile perché `ihold()` non fallisce):

```

    if (inode) {
-       event->inode = igrab(inode);
-       if (!event->inode)
-           goto lost_event;
+       ihold(inode);
+       event->inode = inode;
    }
    ...
-lost_event:
-    fserror_free_event(event);
lost:
    if (inode)
        pr_err_ratelimited(...);

```

Sette righe cancellate, due aggiunte, zero modifiche al contratto pubblico della funzione. La label `lost_event` e il suo error path vengono rimossi completamente: senza `igrab()` non esiste più un caso in cui un evento già allocato debba essere liberato per fallimento di acquisizione del riferimento, quindi la label è codice morto.

8 Riconfigurazione del kernel e rebuild

Prima di testare il fix è stata effettuata una riconfigurazione mirata del kernel: disabilitazione di KASAN. La motivazione è sia tecnica sia pragmatica.

Tecnicamente, KASAN e lockdep sono due sanitizer completamente indipendenti. KASAN rileva violazioni di memoria a runtime (use-after-free, out-of-bounds, accessi a memoria avvelenata) ed era lo strumento rilevante per il primo bug del capitolo 3. Lockdep (`CONFIG_PROVE_LOCKING`) rileva inconsistenze nell'ordinamento dei lock ed è lo strumento rilevante per il bug corrente. Il warning che si sta fixando è emesso da lockdep; KASAN non partecipa in alcun modo al path di rilevamento né alla verifica del fix.

La configurazione di syzbot ha KASAN abilitato perché syzbot è un fuzzer continuo che deve rilevare qualsiasi tipo di bug in un singolo run, senza sapere in anticipo se si tratterà di use-after-free, lock violation, out-of-bounds o altro. Attiva tutti i sanitizer contemporaneamente per massimizzare la copertura. Nel contesto di sviluppo mirato, invece, l'obiettivo è noto e la ridondanza costa.

Praticamente, KASAN raddoppia circa la dimensione dei file oggetto e triplica il consumo di RAM del linker: `vmlinux.o` con KASAN richiedeva più degli 8 GB della VM (costringendo al `-j2` e all'aggiunta di 4 GB di swap, come descritto al capitolo 5.3). Con KASAN disabilitato il build torna in territorio gestibile:

```

cd ~/xfs-linux
scripts/config --disable KASAN \
               --disable KASAN_GENERIC \
               --disable KASAN_INLINE \
               --disable KASAN_OUTLINE
make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- olddefconfig
make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- -j$(nproc) bzImage

```

Con `-j$(nproc)` pieno (8 core), senza più andare in OOM, il build si completa nell'ordine dei minuti anziché della ventina di minuti del primo tentativo. `CONFIG_PROVE_LOCKING` resta abilitato: è l'unica opzione strettamente necessaria per la verifica del fix.

9 Test di controllo: baseline senza fix

Prima di applicare la modifica al codice, è stato eseguito un **test di controllo** sul kernel ricompilato con la nuova configurazione (lockdep sì, KASAN no) ma **senza** il fix. L'obiettivo è isolare la variabile sotto esame: se il warning sparisse anche senza il fix, non si potrebbe concludere nulla dall'eventuale assenza del warning nel test successivo. Il test di controllo deve quindi riprodurre il warning.

Il flusso di lavoro è stato il seguente. La modifica a `fs/fserror.c` era già stata sviluppata e committed in precedenza sul branch `for-next`; per il test di controllo è stato sufficiente riportare il working tree allo stato non-patchato tramite `git stash`:

```
cd ~/xfs-linux
git stash
git diff fs/fserror.c # output vuoto: file tornato all'originale
make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- -j$(nproc) bzImage
```

Il kernel risultante ha quindi le stesse opzioni di debug del test finale (lockdep attivo, KASAN disattivo) ma mantiene il codice originale con `igrab()`.

L'avvio di QEMU con questo kernel e l'esecuzione del reproducer ha confermato la riproduzione del warning con la stessa firma osservata nei test precedenti: `inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage su i_lock_key#34` alla chiamata `igrab+0x2a`, stesso call trace `bio_endio → iomap_read_end_io → iomap_finish_folio_read → fserror_report → igrab`.

Questo conferma due cose. Primo, disabilitare KASAN non ha effetto sulla riproducibilità del bug: il warning è una proprietà del path di lock, non una conseguenza dell'instrumentazione KASAN. Secondo, il bug è presente anche con il rebuild più veloce (`-j$(nproc)` senza swap aggiuntivo), quindi la configurazione di test è pienamente operativa per la verifica del fix.

10 Applicazione del fix e verifica

Terminato il test di controllo, le modifiche sono state riapplicate con `git stash pop` e il kernel ricompilato:

```
cd ~/xfs-linux
git stash pop
git diff --stat fs/fserror.c
# fs/fserror.c | 7 +-----
# 1 file changed, 2 insertions(+), 5 deletions(-)
make ARCH=x86_64 CROSS_COMPILE=x86_64-linux-gnu- -j$(nproc) bzImage
```

Il build è incrementale: solo `fs/fserror.c` viene ricompilato, seguito dal re-linking di `vmlinux` e dalla generazione di `bzImage`. L'intera operazione richiede alcune decine di secondi.

Il kernel patchato è stato avviato con lo stesso comando QEMU utilizzato per tutti i test precedenti. Il reproducer è stato eseguito con la stessa modalità del protocollo 20/20. L'output seriale mostra il reproducer che avanza attraverso le sue iterazioni di mount/unmount e di I/O error, ma il blocco **WARNING: inconsistent lock state non appare**. Il filesystem XFS continua a segnalare I/O error come previsto (il reproducer fornisce un'immagine XFS volutamente corrotta), la softirq di completamento bio continua a essere servita, `iomap_finish_folio_read` e `fserror_report` vengono ancora chiamate — ma la chiamata a `ihold()` non acquisisce `inode->i_lock`, quindi lockdep non osserva più la combinazione `SOFTIRQ-ON-W ↔ IN-SOFTIRQ-W` e non emette alcun warning.

Il comportamento osservato è esattamente quello atteso: lo stesso codice che prima triggerava il warning ora lo evita, senza alterare le operazioni osservabili (segnalazione errori, notifica `fsnotify`, accounting del reference count dell'inode). Il reproducer è stato lasciato iterare per tutta la sua durata e successivamente rilanciato, senza mai produrre output lockdep.

Combinato con il test di controllo, il risultato è una verifica per differenza: stesso kernel, stesso ambiente, stesso reproducer, stessa configurazione, unica variabile isolata = il contenuto di `fs/fserror.c`. La scomparsa del warning è attribuibile al fix.

11 Analisi della preconditione: debug e confronto `igrab/ihold`

In seguito al meeting del 16 aprile, il Prof. Valente ha richiesto di approfondire i frammenti di codice coinvolti e, in particolare, di rispondere alla domanda centrale: *«quando viene chiamata `fserror_report()`, si può essere certi che `i_count >= 1`?»*. Questa è la preconditione necessaria affinché `ihold()` possa sostituire `igrab()` in modo sicuro: `ihold()` non esegue nessun check su `I_FREEING` e presuppone che il caller detenga già un riferimento valido.

Per rispondere empiricamente sono stati aggiunti punti di debug al codice del kernel, raccogliendo evidenza a runtime a complemento dell'analisi teorica.

11.1 Strumentazione di debug

In `fs/inode.c`, alla funzione `iput_final()`: un messaggio che registra ogni transizione a `I_FREEING`, tracciando il valore di `i_count` al momento della transizione:

```
pr_warn("iput_final [DBG]: ino=%lu i_count=%d -> setting I_FREEING"
        " <- I_FREEING only here, only at i_count=0\n",
        inode->i_ino, atomic_read(&inode->i_count));
```

In `fs/fserror.c`, alla funzione `fserror_report()`: un messaggio di ingresso che registra, per ogni chiamata, `i_count`, `i_state`, i flag `I_FREEING/I_WILL_FREE`, il contesto di esecuzione e il PID; un secondo messaggio dopo l'incremento che registra `i_count` post-`igrab`.

In `fs/iomap/buffered-io.c`, alla funzione `iomap_finish_folio_read()`: un messaggio che registra i valori dell'inode al momento della chiamata a `fserror_report_io()`, come punto di osservazione upstream.

In `fs/inode.c`, alla funzione `unlock_new_inode()`: un messaggio che registra ogni acquisizione della lock class in process context, identificando i punti di registrazione dello stato `SOFTIRQ-ON-W`. Per evitare flooding, questo messaggio usa un pattern `atomic_cmpxchg` che lo stampa una sola volta per boot.

11.2 Evidenza runtime: `log_debug.txt`

Il log di esecuzione con la strumentazione attiva è disponibile nel file `log_debug.txt` (circa 3000 righe). L'analisi delle 22 chiamate tracciate a `fserror_report()` produce i risultati sintetizzati di seguito (dettagliati in `stats_log.txt`):

Metrica	Valore
Chiamate totali a <code>fserror_report()</code> tracciate	22
Da contesto <code>softirq</code> (<code>in_softirq=256</code>)	4
Da contesto processo (<code>in_softirq=0</code>)	18
<code>i_count</code> all'ingresso = 1	19
<code>i_count</code> all'ingresso = 2 (chiamate concorrenti)	3
<code>I_FREEING != 0</code> all'ingresso	0 / 22
<code>I_WILL_FREE != 0</code> all'ingresso	0 / 22
<code>i_count</code> dopo <code>igrab</code> osservato ≥ 2	22 / 22
<code>igrab()</code> ha mai ritornato <code>NULL</code>	mai

In tutti e 22 i casi, `i_count  $\geq 1$` e `I_FREEING == 0` al momento dell'ingresso in `fserror_report()`. `igrab()` non ha mai ritornato `NULL`. Le 3 chiamate con `i_count=2` all'ingresso (call #2, #4, #6) sono chiamate concorrenti: una seconda istanza del reproducer stava già tenendo una reference quando la prima ha chiamato `igrab`.

Il log `iput_final [DBG]` fornisce la prova della proprietà duale: in tutte le occorrenze, `I_FREEING` viene settato **esclusivamente** quando `i_count == 0`. Le due condizioni `{i_count  $\geq 1$ }` e `{I_FREEING == 0}` sono quindi mutuamente esclusive per costruzione del codice, non per osservazione.

11.3 Confronto `igrab` vs `ihold`: espansione assembly

Il documento allegato `igrab-vs-ihold.pdf` riporta l'espansione ricorsiva completa delle due funzioni fino all'istruzione assembly su x86-64. Di seguito i risultati principali.

Percorso di `igrab`: `__iget` $\rightarrow$ `atomic_inc` $\rightarrow$ `raw_atomic_inc` $\rightarrow$ `raw_atomic_add(1, v)` [`#else`] $\rightarrow$ `arch_atomic_add`:

```
lock addl $1, i_count(%rdi)
```

Percorso di `ihold`: `atomic_inc_return` $\rightarrow$ `raw_atomic_inc_return` $\rightarrow$ `raw_atomic_add_return(1, v)` [`#else`] $\rightarrow$ `arch_atomic_add_return` $\rightarrow$ `xadd` $\rightarrow$ `__xadd` $\rightarrow$ `__xchg_op`:

```
lock xaddl %eax, i_count(%rdi) ; fetch-and-add atomico: i_count += 1
add $1, %eax ; calcola nuovo valore per WARN_ON
```

L'effetto su `i_count` è identico in entrambi i casi: `i_count += 1`. `XADDL` (exchange-and-add) restituisce in aggiunta il vecchio valore nel registro, necessario affinché `atomic_inc_return` possa ritornare il nuovo valore al `WARN_ON(result < 2)` di `ihold`. Il contatore viene incrementato allo stesso modo.

Una verifica sul sorgente dell'architettura conferma i percorsi: su x86-64 né `arch_atomic_inc` né `arch_atomic_inc_return` né `arch_atomic_inc_return_relaxed` sono definite come funzioni specifiche (verificato con `grep` su `arch/x86/include/asm/atomic*.h`). Entrambi i percorsi cadono nei rami `#else` dei fallback generici `atomic-arch-fallback.h`, che chiamano rispettivamente `arch_atomic_add` e `arch_atomic_add_return`, le uniche effettivamente definite su x86.

La tabella seguente riepiloga cosa cambia permanentemente e cosa no tra le due chiamate:

Operazione	igrab	ihold	Effetto permanente
<code>preempt_disable</code> / <code>enable</code>	sì (si annulla)	no	no — torna al valore originale
<code>spin_lock</code> / <code>spin_unlock</code>	sì (si annulla)	no	no — lock rilasciato prima del return
Interazione con lockdep	sì → può scattare WARNING	no	solo checker, nessun effetto runtime
Lettura di <code>i_state</code>	sì	no	no — sempre falso se <code>i_count >= 1</code>
Incremento di <code>i_count</code>	<code>lock addl \$1</code>	<code>lock xaddl \$1</code>	sì — identico
<code>WARN_ON(result < 2)</code>	no	sì	no — mai preso se <code>i_count >= 1</code>

11.4 Conclusione sull'equivalenza

Al termine di entrambe le chiamate, l'unico stato permanentemente modificato è lo stesso: `i_count += 1`. La sostituzione `igrab()` → `ihold()` è:

- **Corretta:** la preconditione di `ihold()` è verificata empiricamente in 22/22 chiamate e giustificata teoricamente dalla presenza del folio in page cache durante l'I/O in flight.
- **Sicura:** `ihold()` non acquisisce nessun lock, è safe da qualsiasi contesto.
- **Equivalente:** produce lo stesso incremento atomico di `i_count`, indistinguibile dal punto di vista del runtime.

12 Generazione della patch e controlli di stile

Verificata la correttezza funzionale, la patch è stata preparata per la sottomissione. Il commit sul branch `for-next` è:

```
luffy@evil-corp:~/xfs-linux$ git log --oneline -3
00df63ea80c2 (HEAD -> for-next) fs/fserror: use ihold() instead of
                                igrab() to fix softirq lockdep warning
```

```
2ffc6900d5c3 (origin/xfs-7.1-merge, origin/for-next) xfs: fix number
              of GC bvecs
9de45faed34d xfs: untangle the open zones reporting in mountinfo
```

Il commit 00df63ea80c2 è un singolo commit pulito sopra origin/for-next, con working tree pulito (git status riporta «nothing to commit, working tree clean»). Il formato-patch è stato generato con:

```
git format-patch -1 --signoff -o /tmp/
```

Il contenuto della patch generata:

```
From 00df63ea80c26a7b49660100039348f779f79137 Mon Sep 17 00:00:00 2001
From: Francesco Caligiuri <fracali99@gmail.com>
Date: Thu, 16 Apr 2026 14:10:01 +0200
Subject: [PATCH] fs/fserror: use ihold() instead of igrab() to fix
        softirq lockdep warning

fserror_report() calls igrab(inode) which acquires inode->i_lock via
spin_lock(). This is unsafe when called from softirq context, because
the same lock class (i_lock_key) is also acquired from process context
with softirqs enabled during XFS mount (unlock_new_inode -> xfs_iget ->
xfs_mountfs), creating an inconsistent lock state detected by lockdep:

    inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage
    ksoftirqd takes: (&sb->s_type->i_lock_key#34), at: igrab+0x2a/0x230

The path that triggers this is:
    bio_endio -> iomap_read_end_io -> iomap_finish_folio_read ->
    fserror_report -> igrab

Replace igrab() with ihold(). ihold() increments i_count atomically
without acquiring any lock, making it safe from any context.

This is correct because callers of fserror_report() with a non-NULL
inode must already hold a valid reference to that inode. In the iomap
read-completion path, the page cache holds a folio reference for the
duration of the in-flight bio, ensuring the inode cannot be in the
I_FREEING state. The I_FREEING check performed by igrab() is therefore
unnecessary.

Fixes: 21945e6cb516 ("fs: report filesystem and file I/O errors to
        fsnotify")
Reported-by: syzbot+5eb0d61dfb76ca12670c@syzkaller.appspotmail.com
Closes: https://syzkaller.appspot.com/bug?extid=5eb0d61dfb76ca12670c
Signed-off-by: Francesco Caligiuri <fracali99@gmail.com>
---
fs/fserror.c | 7 ++-----
1 file changed, 2 insertions(+), 5 deletions(-)
```

Il diff della patch è quello già descritto al capitolo 7: sostituzione di igrab() con ihold() + rimozione del label lost_event.

Il messaggio di commit segue le convenzioni della comunità kernel per quanto riguarda i trailer. **Fixes:** punta al commit 21945e6cb516 ("fs: report filesystem and file I/O errors to fsnotify"), il commit che ha introdotto `fserror_report()` con la chiamata a `igrab()` originale — questo trailer permette il backport automatico sulle versioni stabili in cui il codice difettoso è presente. **Reported-by:** e **Closes:** legano la patch al bug report syzbot, in modo che syzbot possa correlare il fix e chiudere automaticamente il bug quando la patch raggiunge `linux-next`. **Signed-off-by:** è il Developer Certificate of Origin richiesto da tutte le contribuzioni al kernel.

12.1 Verifica con `checkpatch.pl`

Lo script `scripts/checkpatch.pl` del kernel applica le regole di stile e coerenza del `Documentation/process/coding-style.rst` alla patch. È stato eseguito in modalità `--strict` (più severa del livello `standard`):

```
luffy@evil-corp:~/xfs-linux$ scripts/checkpatch.pl --strict \
                               /tmp/0001-fs-fserror-*.patch
total: 0 errors, 0 warnings, 0 checks, 19 lines checked

/tmp/0001-fs-fserror-*.patch has no obvious style problems and
is ready for submission.
```

Zero errori, zero warning, zero check informativi. La patch rispetta coding style, indentazione, line length e convenzioni dei trailer.

12.2 Identificazione dei maintainer

Lo script `scripts/get_maintainer.pl` analizza il patch, consulta il file `MAINTAINERS` e produce la lista dei destinatari corretti per la sottomissione via email:

```
luffy@evil-corp:~/xfs-linux$ scripts/get_maintainer.pl \
                               /tmp/0001-fs-fserror-*.patch
Alexander Viro <viro@zeniv.linux.org.uk>
    (maintainer:FILESYSTEMS (VFS and infrastructure))
Christian Brauner <brauner@kernel.org>
    (maintainer:FILESYSTEMS (VFS and infrastructure),
    blamed_fixes:1/1=100%)
Jan Kara <jack@suse.cz>
    (reviewer:FILESYSTEMS (VFS and infrastructure),
    blamed_fixes:1/1=100%)
"Darrick J. Wong" <djwong@kernel.org>
    (blamed_fixes:1/1=100%)
linux-fsdevel@vger.kernel.org
    (open list:FILESYSTEMS (VFS and infrastructure))
linux-kernel@vger.kernel.org
    (open list)
```

La destinazione corretta è quindi un `git send-email` ai tre maintainer VFS (Viro, Brauner, Kara), con in CC Darrick J. Wong (maintainer XFS, tecnicamente blamed per il commit che ha introdotto la chiamata originale a `igrab`) e le due mailing list `linux-fsdevel` e `linux-kernel`. Il flag `blamed_fixes:1/1=100%` indica che il tool, analizzando il trailer **Fixes:** della patch,

conferma che queste persone sono quelle che hanno scritto o revisionato il codice introdotto originariamente.

13 Stato finale e prossimi passi

Il lavoro ha completato il ciclo previsto dal progetto: selezione del bug, riproduzione consistente, analisi del call trace, progettazione del fix, applicazione, verifica per differenza, generazione della patch formale con passaggio pulito di `checkpatch.pl --strict` e identificazione dei maintainer.

Il commit `00df63ea80c2` è pronto sul branch locale `for-next` e il file patch `0001-fs-fserror-use-ihold-instead-of-igrab-to-fix-softirq.patch` è pronto per la sottomissione alla mailing list kernel.

Il meeting di revisione con il Prof. Valente si è svolto il 16 aprile alle 16:30. In seguito al meeting, è stata condotta l'analisi approfondita documentata nel capitolo 11 (strumentazione di debug, raccolta del log di esecuzione, analisi statistica delle 22 chiamate, espansione assembly di `igrab` e `ihold`), che fornisce evidenza empirica e teorica completa a supporto della correttezza del fix. I materiali prodotti (`log_debug.txt`, `stats_log.txt`, `igrab-vs-ihold.pdf`) sono stati inviati al docente il 17 aprile.

Il passo successivo è la sottomissione della patch alla mailing list `linux-fsdevel`, da effettuare dopo la validazione finale da parte del docente. Eventuali modifiche emerse dalla revisione saranno incorporate come versione v2 della patch prima dell'invio.

14 Riepilogo del lavoro svolto

Fase	Stato
Primo bug: selezione <code>fserror_worker</code> UAF (syzkaller)	✓ Completata
Setup ambiente: VM ARM64 Debian su UTM + QEMU x86_64	✓ Completato
Workaround cross-compile: uso degli asset pre-compilati di syzbot	✓ Completato
Primo bug: riproduzione del crash KASAN in QEMU	✓ Completata
Primo bug: analisi KASAN e root cause (<code>xfi_iget</code> → <code>call_rcu</code> diretto)	✓ Completata
Primo bug: patch tentative (<code>WARN</code> + guard a <code>lost_event</code>)	✓ Completate, incomplete
Decisione di cambio bug per riproducibilità insufficiente	✓ Documentata
Nuovo bug: selezione <code>inconsistent lock state</code> in <code>igrab</code>	✓ Completata
Nuovo bug: riproduzione iniziale (3/4) con asset syzbot	✓ Completata
Test sistematico 20 boot (19/20 = 95%)	✓ Completato
Analisi meccanismo lockdep one-shot (<code>usage_mask</code>)	✓ Completata
Lettura e interpretazione del call trace lockdep	✓ Completata
Compilazione del kernel <code>for-next</code> self-compiled	✓ Completata
Riproduzione del bug sul kernel self-compiled	✓ Completata
Progettazione del fix (<code>igrab</code> → <code>ihold</code>)	✓ Completata

Fase	Stato
Riconfigurazione kernel senza KASAN e rebuild	✓ Completata
Test di controllo: baseline senza fix (warning presente)	✓ Completato
Applicazione del fix e verifica (warning assente)	✓ Completata
Commit sul branch <code>for-next</code> con DCO Signed-off-by	✓ Completato
<code>git format-patch -1 --signoff</code> generato	✓ Completato
<code>checkpatch.pl --strict</code> : 0 errors / 0 warnings / 0 checks	✓ Passato
<code>get_maintainer.pl</code> : lista destinatari identificata	✓ Completata
Meeting di revisione con il docente (16 apr 16:30)	✓ Svolto
Strumentazione debug: <code>inode.c</code> , <code>fserror.c</code> , <code>buffered-io.c</code>	✓ Completata
Raccolta log di debug (<code>log_debug.txt</code> , 3000 righe)	✓ Completata
Analisi statistica 22 chiamate (<code>stats_log.txt</code>)	✓ Completata
Espansione assembly <code>igrab</code> vs <code>ihold</code> (<code>igrab-vs-ihold.pdf</code>)	✓ Completata
Invio allegati al docente (log, stats, PDF)	✓ Completato
Sottomissione alla mailing list VFS (<code>linux-fsdevel</code>)	Post-revisione docente

15 Riferimenti

- Syzbot — bug corrente: <https://syzkaller.appspot.com/bug?extid=5eb0d61dfb76ca12670c>
- Syzbot — primo bug (abbandonato): <https://syzkaller.appspot.com/bug?extid=fbf6ff30de890ff32ec5>
- Lockdep design: <https://www.kernel.org/doc/html/latest/locking/lockdep-design.html>
- KASAN documentation: <https://www.kernel.org/doc/html/latest/dev-tools/kasan.html>
- RCU documentation: <https://www.kernel.org/doc/html/latest/RCU/whatisRCU.html>
- Syzkaller: <https://github.com/google/syzkaller>
- XFS documentation: <https://www.kernel.org/doc/html/latest/filesystems/xfs/index.html>
- Kernel development process: <https://www.kernel.org/doc/html/latest/process/development-process.html>
- Elixir Bootlin (cross-reference kernel): <https://elixir.bootlin.com/linux/>